Operating Systems Laboratory

Laboratory Assignment 5

Parallel Applications

Threads, Pipes, Shared Memory and Sockets

Submitted by

Ayush Yadav – CS24BT001

Trisham Bepari – CS24BT022

Abstract. The image sharpener of Lab 1 (S1 smoothen, S2 find details, S3 sharpen) was rebuilt as a pipeline whose stages run on different cores and pass their pixels to each other, in four ways: three threads with a spin lock made of atomic operations, three processes with pipes, three processes with shared memory and semaphores, and two programs connected by a TCP socket. Every version writes exactly the image that Lab 1 wrote, for all seven Lab 1 images (143 outputs compared, none different), and a pixel count with an order-dependent hash at both ends of every link proves that the pixels are received as sent, in the sent order. Passing every pixel on its own makes all versions 16 to 28 times *slower* than the sequential baseline: handing a pixel to another core costs 0.5 to 1 μs, computing it 35 ns. Passing fewer results (variant 2), or the same pixels in larger messages, gives speed-ups of 1.1 to 1.6. The limit is 1.61 and not 3, because S1 is 62 % of the work and a pipeline is as fast as its slowest stage. For the same reason, giving S1 a computer of its own is the better of the two partitions in Part III. Two performance problems that the source code does not show were found and fixed on the way: false sharing of a cache line, and the address at which the linker places a function.

November 12, 2026

1 Setup and Design

The application. The image sharpener of Lab 1 has three stages. **S1** (smoothen) replaces a pixel by the average of its 3×3 neighbourhood, **S2** (find details) computes $\max(0,\ \text{input} - \text{smoothened})$ and **S3** (sharpen) computes $\min(255,\ \text{input} + \text{details})$, each on the three colour channels. In this assignment the three stages run at the same time on different cores (Part II) or on two computers (Part III), as a pipeline: a stage does not wait for the previous stage to finish the image, it works on every pixel as soon as that pixel arrives.

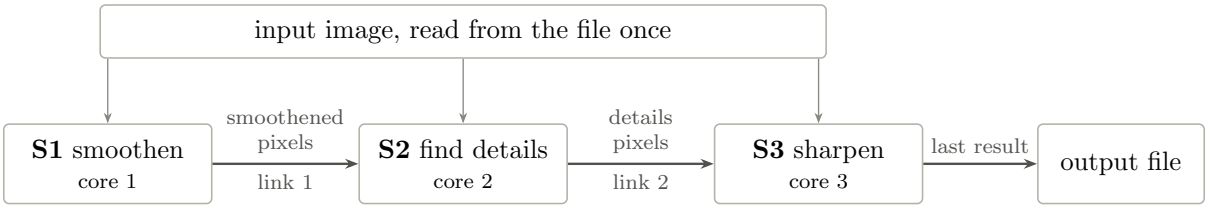


Figure 1: The pipeline. Every stage can read the input image; only the results travel over the two links.

What travels over a link. Only the result of a stage: 3 bytes per pixel, in row-major order (left to right, top to bottom). No coordinates are sent; the receiver knows which pixel it got only from its position in the stream. This is why the order matters, and why Section 4 proves that it is kept. All stages also need the input image. Threads share it, forked processes each get a copy from `fork()`, and in Part III computer A sends it to computer B once at the start, so the image file is read exactly once in every program.

Messages. A link carries *messages* of `BATCH` pixels. The default is `BATCH=1`: every pixel is passed on the moment it is ready, exactly as the assignment asks. A larger value groups pixels (`BATCH=450` is one row of our image), which is used in Section 5 to study the cost of communication. In both cases the next stage starts long before the previous one has finished the image.

The two variants. The transformation is repeated 1000 times on the image that was read once, to make the experiment compute-intensive.

	Runs of each stage	Results passed per link	Pixels passed per link
Variant 1	1000	1000 (every run)	$1000 \times 81\,000 = 81$ million
Variant 2	1000	10 (every 100th run)	$10 \times 81\,000 = 810\,000$

Table 1: The two variants do the same amount of computation; variant 2 communicates 100 times less.

In variant 2, S1 passes on only its 100th, 200th, ... run. S2 runs 100 times on every result it gets: its first run takes the smoothened pixels from the link while S1 is still producing them, runs 2–100 reuse the copy that S2 kept, and the pixels of the 100th run are passed to S3, which works the same way. The code is the same for both variants; a stage receives in run r if $(r-1) \bmod g = 0$ and passes its result on if $r \bmod g = 0$, with $g = 1$ in variant 1 and $g = 100$ in variant 2. In both variants the image written to the file is the result of the last run of S3, and since every run starts from the same input image, it must be the image that Lab 1 produced.

Programs. One self-contained program per part (Table 2). All of them use the same three functions that process one pixel (`S1_smoothen_pixel`, `S2_find_details_pixel`, `S3_sharpen_pixel`), which are the loop bodies of the Lab 1 functions, unchanged.

Program	Workers	Link	Synchronization
`part1`	one thread	–	–
`part2_1`	3 threads of one process	queue in the address space	spin lock (`atomic_flag::test_and_set`)
`part2_2`	3 processes (`fork`)	pipe	by the kernel, in `read`/`write`
`part2_3`	3 processes (`fork`)	queue in shared memory	two counting semaphores per queue
`part3_1`	A: S1+S2, B: S3	TCP socket	by the kernel, in `read`/`write`
`part3_2`	A: S1, B: S2+S3	TCP socket	by the kernel, in `read`/`write`

Table 2: The programs. Each takes the variant (1 or 2), and optionally the number of runs and `BATCH`.

Measurements. All runs were done on an Intel Core i7-8665U laptop (4 cores, 8 hardware threads), Ubuntu 24.04, `g++ -O2 -falign-functions=64` (the second flag is explained in Section 5.1). The input is image 1 of Lab 1 (450×180, 81 000 pixels) unless stated otherwise. Times are measured inside the programs with `chrono::steady_clock`. The *processing time* runs from the moment the stages are started to the moment S3 has finished its last run; reading and writing the files is reported separately. Every number is the median of 3 to 9 runs, and

$$\text{speed-up} = \frac{\text{processing time of the baseline}}{\text{processing time of the parallel program}}.$$

Building and running. From `src/`; every target compiles its program and runs it:

```
make part1                                  # creates output_part1.ppm
make part2_1_variant1                       # also part2_2, part2_3 and variant2
make part2_2_variant1 BATCH=450 RUNS=1000   # optional: pixels per message, runs
make part3_1_variant1_A                     # computer A (waits for B on PORT=12345)
make part3_1_variant1_B HOST=<address of A> # computer B (also part3_2, variant2)
make compare                                # compares all outputs with Lab 1
```

2 Part I: Baseline

`part1` is the Lab 1 program with two changes that are needed to run the transformation 1000 times. First, the Lab 1 functions allocated a new image on every call (one `new` per pixel); the three images are now allocated once and the functions fill them. Second, the body of each loop was moved into a function that handles one pixel, so that the parallel programs can call exactly the same code. The arithmetic is untouched, and with one run the output is identical to the Lab 1 output. (One line was also added to `libppm.cpp`: `read_ppm_file` accumulated the width and height in fields that it never set to zero.)

Stage	Time for 1000 runs (s)	Range of 8 runs (s)	Per pixel (ns)	Share
S1 smoothen	1.78	1.63–2.33	22.0	62 %
S2 find details	0.53	0.44–0.96	6.6	18 %
S3 sharpen	0.56	0.48–0.90	6.9	20 %
Processing time (S1 + S2 + S3)	2.86	2.58–4.18	35.3	100 %

Table 3: Baseline, image 1: where the time goes. Median of 8 runs; two of them were disturbed by other activity on the laptop, which is why the upper end of the range is far from the median.

What a pipeline can gain. Smoothening is 62 % of the work, because it reads nine pixels and divides three times for every pixel it produces, while S2 and S3 need one subtraction or addition per channel. This limits what three cores can do. The stages run at the same time, but every pixel still has to pass through S1, and S1 has one core. A pipeline is as fast as its slowest stage, so with free communication

$$\text{speed-up} \;\le\; \frac{t_{S1} + t_{S2} + t_{S3}}{\max(t_{S1},\, t_{S2},\, t_{S3})} = \frac{2.86}{1.78} = 1.61,$$

not 3. The same reasoning gives the limits for the two partitions of Part III: the pipeline is as fast as the more loaded computer, so *S1+S2 | S3* can reach at most $2.86/(1.78 + 0.53) = 1.24$ and *S1 | S2+S3* at most $2.86/1.78 = 1.61$. These limits are drawn in the graphs of Section 5.

3 Part II: One Pipeline, Three Mechanisms

The three programs have the same structure. A stage loops over its runs and over the pixels of the image; it calls a *receive* function when it reaches a pixel that it does not have yet, processes the pixel, and calls a *send* function as soon as a message is complete. Only these two functions, and the way the three workers are created, differ between the programs.

3.1 Threads and atomic operations (`part2_1`)

The three stages are three `std::thread`s. A link is a queue in the address space of the process: a circular buffer of 64 message slots with the fields `head`, `tail` and `count`. The fields are protected by a spin lock made of one `atomic_flag`. `test_and_set()` sets the flag and returns the value it had before, in one atomic step, so it returns `false` for exactly one thread: the one that changed the flag from clear to set. That thread owns the lock until it calls `clear()`.

```
void send_pixels(struct queue_t *queue, uint8_t *pixels, int number_of_pixels)
{
    while (true)
    {
        while (queue->lock.test_and_set()) {}       // spin until the lock is ours
        if (queue->count < QUEUE_SIZE)
        {
            memcpy(queue->messages + (size_t) queue->tail * batch * 3,
                   pixels, number_of_pixels * 3);
            queue->tail = (queue->tail + 1) % QUEUE_SIZE;
            queue->count++;
            queue->lock.clear();
            return;
        }
        queue->lock.clear();                        // queue full: let the receiver in, try again
    }
}
```

`receive_pixels` is the mirror image (it waits while `count` is 0). Nothing ever sleeps: a stage that waits for a pixel keeps taking and releasing the lock, so it keeps a core fully busy. No other synchronization is used.

A false-sharing bug. The first version passed every test but was strangely slow: variant 2, which passes only 10 results, took 3.6–4.4 s, much longer than the baseline. The symbol table (`nm`) showed why: the linker had placed the first queue, with its lock, in the same 64-byte cache line as the read-only globals `input_image`, `batch` and `runs`. The waiting threads write the lock continuously, so S1 lost that cache line again and again while it computed, although it only *reads* those globals. Declaring the queue as `struct alignas(64) queue_t` gives each queue a cache line of its own and brought the same run down to 2.1–2.2 s. Nothing was wrong with the logic; the program was only slow.

3.2 Processes and pipes (`part2_2`)

The parent reads the image, creates two pipes with `pipe()` and calls `fork()` three times; it then only waits for its children and checks their exit status. A child sends a message with one `write()` and receives it with `read()`. The kernel does all the synchronization: `read()` blocks while the pipe is empty and `write()` blocks while it is full (a pipe holds 64 KB by default, about 21 845 pixels), and a blocked process sleeps without using a core. Three details needed care:

- *Unused ends must be closed* in every process. `read()` reports the end of the stream only when *all* copies of the write end are closed, and `fork()` copies all four descriptors into all three children.
- *Short reads and writes.* `read()` returns as soon as some bytes are available and `write()` may write only part of a large message, so both are wrapped in loops (`receive_bytes`, `send_bytes`) that continue until the whole message is through.
- *Anonymous pipes are enough* because the three processes have a common parent. Unrelated processes would need a FIFO (`mkfifo`), which behaves in the same way once both ends are open.

3.3 Processes, shared memory and semaphores (`part2_3`)

Before forking, the parent creates a POSIX shared memory object (`shm_open`, sized with `ftruncate`) and maps it with `mmap(..., MAP_SHARED, ...)`. The children inherit the mapping, so all four processes see the same bytes. The name is removed with `shm_unlink` right after the mapping is made: the memory lives on until the last process unmaps it, and nothing is left behind in `/dev/shm` even if the program is killed. The object holds two queues like those of `part2_1`, but each is synchronized by two unnamed semaphores that are stored in the shared memory itself (`sem_init` with `pshared = 1`): `empty_slots` starts at 64 and `full_slots` at 0.

```
void send_pixels(struct queue_t *queue, uint8_t *pixels, int number_of_pixels)
{
    sem_wait(&queue->empty_slots);                  // sleeps while the queue is full
    memcpy(queue->messages + (size_t) queue->tail * batch * 3,
           pixels, number_of_pixels * 3);
    queue->tail = (queue->tail + 1) % QUEUE_SIZE;
    sem_post(&queue->full_slots);                   // wakes the receiver if it sleeps
}
```

`receive_pixels` is the mirror image: it waits on `full_slots`, copies the message at `head` out, and posts `empty_slots`. This is the bounded buffer from the lectures without its mutex. The mutex is not needed here because a queue has exactly one sender and one receiver: `tail` is touched only by the sender and `head` only by the receiver, and the semaphores guarantee that the two never work on the same slot.

4 Proof that Pixels are Received as Sent, in the Sent Order

Method. On every link, the sender and the receiver each keep a count of the pixels and an order-dependent hash of them, the FNV-1a style hash that Lab 4 used for frame numbers. For every pixel p (its three bytes packed into one number), in the order in which it is sent, respectively received:

$$H \leftarrow (H \oplus p) \times 1099511628211 \bmod 2^{64}, \qquad H_0 = 1469598103934665603.$$

When a sender has finished, it hands its count and hash to the receiver's side, and the two pairs are compared. The sender's pair travels outside the path under test or after it: through the pipe or socket *after* the last pixel, through fields in the shared memory, or through global variables. Every program prints, for every link, both pairs and `PASSED` or `FAILED`, and returns a non-zero exit status on a failure.

Why this is a proof. The receiver hashes the pixels in the order in which *it* gets them. The hash is a chain: each step multiplies the result of all previous steps, so swapping two different pixels, changing one bit, losing a pixel or delivering one twice changes the final value (two different sequences collide with a probability of about 2^{-64}). A lost or duplicated pixel also changes the count. Equal counts and equal hashes therefore mean that the receiver got the same sequence of pixels that the sender sent.

Result. The check passed in every run we made: in all runs of Section 8 and in all timed runs of Section 5, for every link. Table 4 shows the default configuration. The five programs agree with each other as well: the stream of smoothened pixels has the same hash whether it goes through a queue, a pipe, shared memory or a socket, and so has the stream of details pixels.

Stream	Variant	Sent = received	Hash at sender = at receiver	Same value in
smoothened (link 1)	1	81 000 000	12170038745284060307	`part2_1`, `2_2`, `2_3`, `3_2`
smoothened (link 1)	2	810 000	16510668365919690495	`part2_1`, `2_2`, `2_3`, `3_2`
details (link 2)	1	81 000 000	1901646794791207459	`part2_1`, `2_2`, `2_3`, `3_1`
details (link 2)	2	810 000	12032347268811867531	`part2_1`, `2_2`, `2_3`, `3_1`

Table 4: Counts and hashes printed by the programs (image 1, 1000 runs, `BATCH=1`).

Does the check notice a fault? A check that always says `PASSED` proves nothing, so we broke the link on purpose in copies of the programs (Table 5).

Fault injected	Link check	Output file
Two neighbouring pixels change places on link 1, once, in the first run (pipes)	`FAILED`: counts equal, hashes differ	identical to Lab 1
One bit of one pixel is flipped on link 1, once, in the first run (pipes)	`FAILED`: counts equal, hashes differ	identical to Lab 1
The spin lock is removed from the queue (threads)	program never finishes (6 of 6 attempts)	not written

Table 5: Fault injection. The first two faults are invisible in the output file.

The first two rows are the reason why comparing the output image with the Lab 1 image is *not* a proof by itself. Only the result of the last run is written to the file, so a pixel that is lost or misplaced in any of the 999 earlier runs leaves no trace there. The hash covers every pixel of every run. The output comparison of Section 8 is still done, as independent evidence for the last run.

5 Run Time and Speed-up

5.1 First, a fair comparison

All programs contain the same function `S1_smoothen_pixel`, so S1 by itself should take the same time in each of them. We checked this with a run in which almost nothing is communicated (variant 2, one whole image per message) and it was *not* true. Built with plain `g++ -O2`, S1 needed about 1.55 s for its 1000 runs in the baseline and in the pipe program, but 2.3–2.5 s in the thread program (Table 6). Left alone, this would have looked like a property of threads. We ruled out the obvious explanations one by one:

- *Busy waiting?* No. When the two waiting threads slept instead of spinning, S1 was just as slow.
- *Hyper-threading or a lower clock?* No. The three threads ran on three different physical cores at about 4.0 GHz, measured during the run.
- *The place of the buffers in memory?* No. Moving S1's buffer with allocator settings changed its address but not the time.

`objdump` then showed that the instructions of the function are identical, byte for byte, in the thread program and in the pipe program. Only the address at which the linker put the function differs. On this processor generation the speed of a piece of code depends on where its jump instructions fall relative to 32-byte boundaries (Intel documents this as the "jump conditional code" erratum), and `S1_smoothen_pixel`, with its nine bounds checks, is mostly jumps. Two things confirmed it:

the assembler option that keeps jumps away from these boundaries removed the difference, and so did `-falign-functions=64`, which starts every function at a 64-byte boundary and therefore gives the function the same layout in every program. All programs are built with the second flag. S1 then takes the same time everywhere, within the run-to-run variation.

Time for 1000 runs of S1 (s), three runs each	`part1`	`part2_1`	`part2_2`	`part2_3`
`g++ -O2`	1.55–1.92	2.28–2.47	1.53–1.77	1.58–1.78
`g++ -O2 -falign-functions=64`	1.69–1.75	1.67–1.79	1.61–1.78	1.73–1.88

Table 6: The same function, placed at different addresses by the linker (top) and at the same position in every program (bottom).

What remains is a run-to-run variation of about ±5 % (it is a laptop: the clock frequency depends on the temperature and on how many cores are busy), so a difference of a few percent between two programs below should not be read as significant.

5.2 Pixel by pixel (`BATCH=1`)

		Variant 1 (81 million pixels per link)				Variant 2 (810 000 per link)		
Program	Workers, link	Time (s)	Range (s)	Speed-up	ns/pix.	Time (s)	Range (s)	Speed-up
`part1`	one thread (baseline)	2.86	2.58–4.18	1	35	2.86	2.58–4.18	1
`part2_1`	threads, atomic operations	67.3	37.1–84.0	0.043	831	2.26	2.12–2.51	1.27
`part2_2`	processes, pipes	56.8	54.9–57.7	0.050	702	2.17	2.00–2.19	1.32
`part2_3`	processes, shared memory	81.5	76.7–82.9	0.035	1006	2.07	2.02–2.17	1.38
`part3_1`	sockets, S1+S2 | S3	45.1	44.1–45.9	0.063	557	2.43	2.33–2.52	1.18
`part3_2`	sockets, S1 | S2+S3	47.0	46.1–47.7	0.061	580	2.22	2.17–2.48	1.29

Table 7: Processing time with every pixel passed on its own (image 1, 1000 runs). Median of 3 runs (variant 1) or 5 runs (variant 2). "ns/pixel" is the time divided by the 81 million pixels that go through the pipeline. The two socket programs ran as two processes on one computer (Section 7).

Variant 1 is 16 to 28 times slower than the baseline. Not one of the five programs gains anything from its extra cores; the fastest of them needs 45 s for what the baseline does in 2.9 s. The reason is in the last column of the left half. The baseline needs 35 ns to take a pixel through all three stages. With every pixel handed to another core on its own, a pixel needs 550 to 1000 ns, however the hand-over is done, so more than 93 % of the time goes into passing pixels and less than 7 % into computing them.

Program	`BATCH`	Elapsed time (s)	CPU time in the program (s)	CPU time in the kernel (s)	Voluntary context switches	Switches per message
`part1`	–	0.30	0.29	0.00	1	–
`part2_1`	1	3.77	11.26	0.01	11	0.000
`part2_2`	1	5.25	2.09	9.46	868 719	0.054
`part2_3`	1	7.14	4.10	11.98	3 250 139	0.201
`part2_1`	450	0.29	0.79	0.01	15	0.000
`part2_2`	450	0.26	0.38	0.07	35 058	0.974
`part2_3`	450	0.30	0.45	0.09	35 392	0.983

Table 8: Variant 1 with 100 runs of image 1 (8.1 million pixels per link, two links), measured with `/usr/bin/time`. A voluntary context switch is a process going to sleep because it has to wait.

Three different ways to lose. Table 8 shows where the time goes when every pixel is a message.

- The *thread* program never enters the kernel and never sleeps (11 voluntary context switches

in the whole run), but it uses 11.3 s of processor time in 3.8 s: all three cores are busy all the time. The baseline needs 0.29 s of processor time for the same computation, so about 97 % of the processor time goes into taking and releasing the two spin locks and into waiting.

- The *pipe* program makes one `write()` and one `read()` per message; `strace` counted 324 043 and 324 037 calls for 324 000 messages. 82 % of its processor time is spent inside the kernel.
- The *semaphore* program needs no system call to pass a message as long as nobody is waiting. But S2 is faster than S1 and keeps catching up with it, so a stage went to sleep 3.25 million times, once every 5 messages. Each time costs a system call to go to sleep, another one to be woken up and a context switch, and in the end 75 % of its processor time is kernel time too.

With one row per message (lower half of the table) the kernel time is almost gone. A stage now goes to sleep for nearly every message (0.97 switches per message), but there are 450 times fewer messages.

Variant 2 is faster than the baseline in every program. It does exactly the same computation but passes 100 times fewer pixels, and the speed-up is 1.18 to 1.38. The two variants were measured with the same programs and the same code path; the only thing that changed is how often the stages talk to each other.

5.3 More pixels per message

Variant 1 does not have to be slow. The stages only have to stop synchronizing once per pixel. With `BATCH`= n a stage collects n pixels and passes them in one message: one lock acquisition, one `write()`, or one pair of semaphore operations for n pixels. The next stage still starts long before the image is finished (an image is 81 000 pixels).

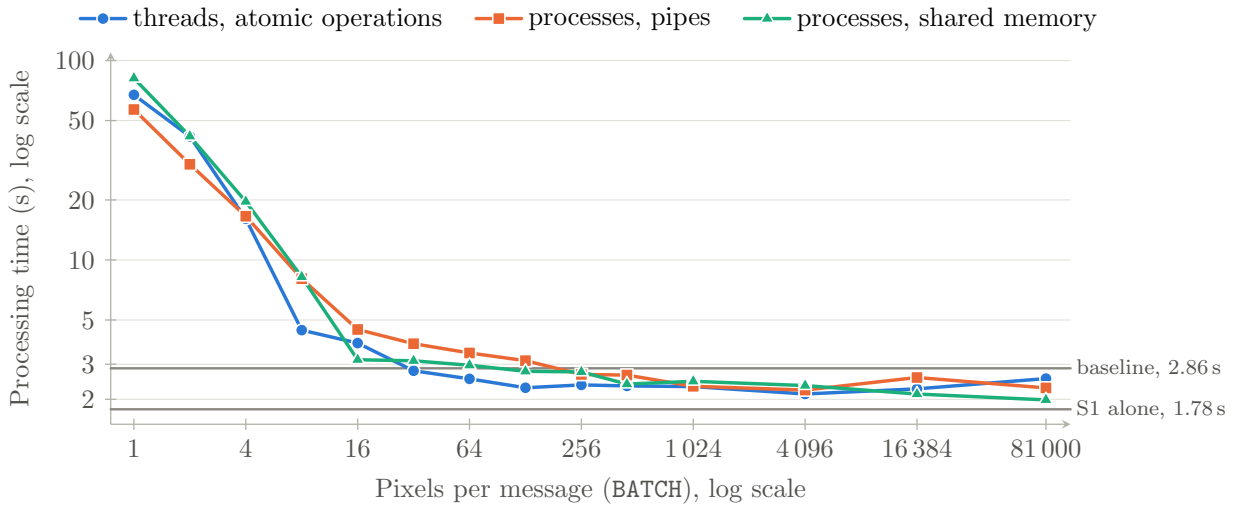


Figure 2: Variant 1 (1000 results per link): processing time against the number of pixels per message. Below the upper grey line a program is faster than the baseline; no pipeline can go below the lower one, the time that S1 needs alone.

		Threads		Pipes		Shared memory	
Pixels per message	Messages per link	Time (s)	Speed-up	Time (s)	Speed-up	Time (s)	Speed-up
1 (one pixel)	81 000 000	67.3	0.043	56.8	0.050	81.5	0.035
2	40 500 000	41.7	0.069	30.2	0.095	41.7	0.069
4	20 250 000	16.1	0.178	16.6	0.173	19.6	0.146
8	10 125 000	4.45	0.64	8.06	0.36	8.22	0.35
16	5 062 500	3.83	0.75	4.48	0.64	3.16	0.91
32	2 531 250	2.78	1.03	3.80	0.75	3.12	0.92
64	1 265 625	2.53	1.13	3.42	0.84	2.96	0.97
128	632 813	2.29	1.25	3.13	0.92	2.76	1.04
256	316 407	2.36	1.21	2.67	1.07	2.74	1.04
450 (one row)	180 000	2.34	1.23	2.65	1.08	2.38	1.20
1 024	79 102	2.32	1.24	2.33	1.23	2.46	1.16
4 096	19 776	2.13	1.35	2.23	1.29	2.34	1.22
16 384	4 944	2.26	1.27	2.57	1.11	2.13	1.35
81 000 (whole image)	1 000	2.54	1.13	2.28	1.25	1.99	1.44

Table 9: The data of Figure 2: variant 1, image 1, 1000 runs, median of 3 runs. The baseline takes 2.86 s.

The time falls in proportion to the number of messages, then stops. From 1 to 8 pixels per message every doubling of the message roughly halves the time (pipes: 56.8, 30.2, 16.6, 8.1 s), which shows that the time was being spent per message and not per pixel. Threads pass the baseline at about 32 pixels per message, shared memory at about 128 and pipes at about 256. The order makes sense: taking a spin lock that nobody else holds costs a few nanoseconds, a semaphore operation costs more and needs the kernel whenever somebody has to be woken up, and a `write()` enters the kernel every time.

Beyond about 1000 pixels per message nothing more is gained. All three programs then take 2.0 to 2.6 s, a speed-up of 1.1 to 1.4, and the differences between them are no larger than the differences between two runs of the same program (for example pipes: 2.23 s at 4096 pixels, 2.57 s at 16 384, 2.28 s for whole images). Communication has become almost free, and what is left is the limit of the pipeline itself: the lower grey line. One row of the image per message (`BATCH=450`) is already in this region.

5.4 Where the limit comes from: variant 2 in detail

Program	`BATCH`	S1 finished after (s)	S2 finished after (s)	S3 finished after (s)	Speed-up
`part2_1`	1	2.19	2.23	2.26	1.27
`part2_2`	1	2.11	2.14	2.17	1.32
`part2_3`	1	2.00	2.04	2.07	1.38
`part2_1`	450	1.89	1.91	1.94	1.47
`part2_2`	450	1.81	1.83	1.86	1.54
`part2_3`	450	1.70	1.72	1.75	1.63

Table 10: Variant 2, image 1, 1000 runs: the time after which each stage had finished (median of 5 runs for `BATCH=1`, of 3 runs for `BATCH=450`). The baseline needs 1.78 s for S1 alone.

The pipeline takes exactly as long as S1. In every row S2 finishes about 0.03 s after S1, and S3 about 0.03 s after S2. That is the time they need for the last result (100 runs take S2 0.05 s). So S2 and S3 are not short of time: each of them has work for about 0.5 s and spends the other 1.2 to 1.7 s waiting for S1. Adding more cores to S2 or S3 would change nothing; only a faster S1 would. With one row per message the programs reach a speed-up of 1.47 to 1.63, which is the limit of 1.61 within the accuracy of the measurement (the baseline's own S1 took between 1.63 and 1.82 s in its undisturbed runs).

Busy waiting is not free. In both halves of Table 10 the thread program is the slowest of the three, by 4 to 11 %, although its stages do the same work. Its S2 and S3 never sleep; they spin while they wait, so three cores are fully busy for the whole run instead of one. A control experiment without any pipeline shows the size of this effect: the baseline's S1 took 1.55 s on a quiet machine, 1.8 s while two unrelated processes were spinning on two other cores (most likely because the processor lowers its clock frequency when more cores are active), and 3.95 s when a spinning process was placed on the second hardware thread of the same core. A stage that waits in `read()` or `sem_wait()` sleeps, and costs the working stage nothing.

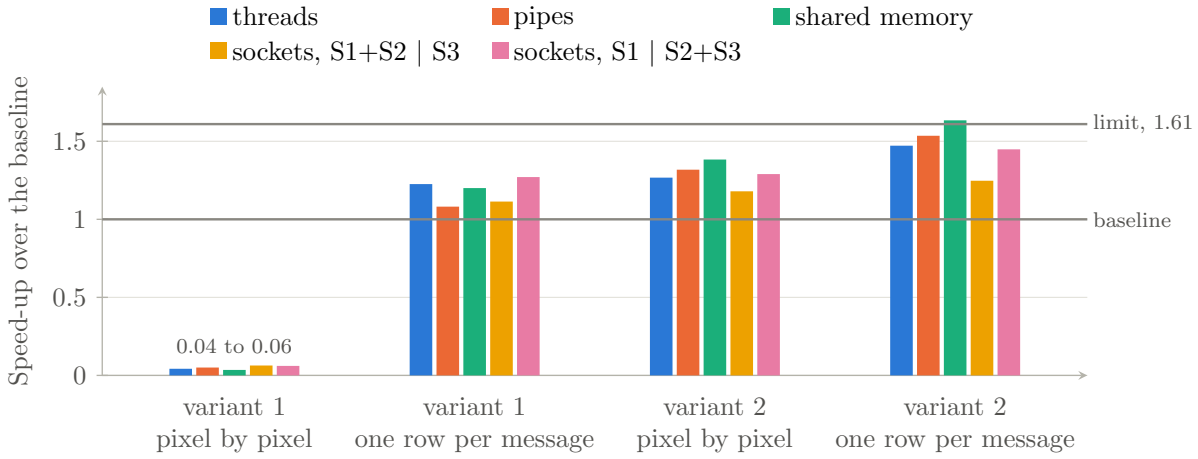


Figure 3: All five programs in four configurations (image 1, 1000 runs; values in Tables 7, 9, 10 and 14). The limit for *sockets, S1+S2 | S3* is 1.24 instead of 1.61.

Summary of the two variants. Figure 3 puts the results side by side. The amount of computation is the same in all four groups. Two things differ: the number of messages per link (81 million, 180 000, 810 000 and 1800) and the number of pixels that are moved at all (81 million in variant 1, 810 000 in variant 2). The first group pays for 81 million synchronizations and loses everything. In the second group the synchronizations are gone, but every pixel is still copied into a message and out of it, and hashed at both ends for the proof; this is why variant 1 stays below variant 2 even with large messages. (Switching the hash off made the three programs of Part II 1 to 8 % faster in the second group, so the proof is the smaller part of that cost.) The picture is the same for threads, pipes, shared memory and sockets.

5.5 Images of other sizes

			Baseline			Speed-up, variant 1			Speed-up, variant 2		
Image	Size	Mpixel	Time (s)	ns/pixel	T/t_{S1}	Thr.	Pipes	Shm	Thr.	Pipes	Shm
1	450 × 180	0.08	0.27	32.7	1.69	1.40	1.23	1.28	1.25	1.29	1.28
2	786 × 393	0.31	1.37	44.4	1.78	1.71	1.49	1.48	1.31	1.28	1.32
3	1024 × 575	0.59	2.72	46.2	1.77	1.71	1.53	1.52	1.34	1.07	1.32
4	1280 × 910	1.16	5.40	46.3	1.82	1.73	1.50	1.51	1.23	1.33	1.32
5	1500 × 1000	1.50	6.94	46.3	1.83	1.69	1.56	1.48	1.21	1.32	1.26
6	2271 × 1500	3.41	15.81	46.4	1.87	1.70	1.44	1.47	1.26	1.27	1.23
7	3750 × 2470	9.26	40.50	43.7	1.93	1.56	1.34	1.38	1.07	1.21	1.24

Table 11: The seven images of Lab 1, 100 runs, one row of the image per message, median of 3 runs (9 for image 1, which was measured separately because its runs last only 0.2 s). With 100 runs variant 2 passes a single result, so its stages work one after the other.

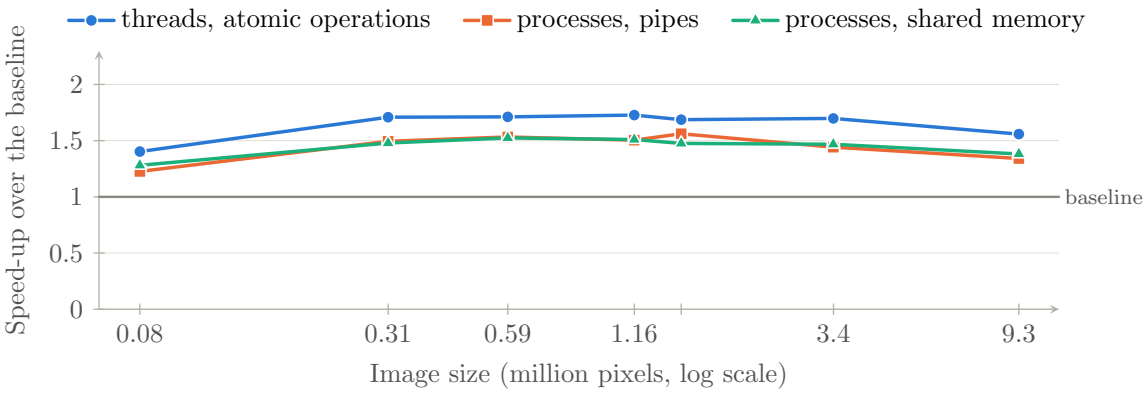


Figure 4: Variant 1, one row per message, 100 runs: speed-up for the seven images.

Larger images give larger speed-ups. For images 2 to 7 the pipeline of variant 1 reaches 1.6 to 1.7 with threads and 1.3 to 1.6 with processes, against 1.2 to 1.4 for image 1. The baseline becomes slower per pixel (44 to 46 ns instead of 33 ns): it keeps four images in which every pixel is a separate 32-byte block, and from image 2 on these no longer fit into the 8 MB cache of the processor. S2 and S3 suffer most, so their share of the work grows and with it the limit T/t_{S1} of the pipeline, from 1.7 to 1.9.

Not all of the speed-up comes from the three cores. The right part of Table 11 is a control experiment. With 100 runs variant 2 passes one single result, so S1, S2 and S3 work one after the other and nothing is done in parallel. Nevertheless these runs are 1.1 to 1.3 times faster than the baseline, for every image. The reason is the data layout: a stage of the pipeline keeps the pixels it receives in a compact array of 3 bytes per pixel, so its S2 is about 2.5 times and its S3 1.2 to 2 times faster than those of the baseline, which follow two pointers for every pixel of every image. A sequential program with compact arrays would gain this factor too. What the three cores add is the difference between the two halves of the table.

Threads are the fastest on large images. A likely reason is in Table 8: with one row per message a sleeping stage is woken up by the kernel for nearly every message, and waking up takes time; a spinning stage takes the row the moment it is ready. Busy waiting costs processor time (Section 5.4) but no latency.

5.6 The original experiment: a single run

		Baseline, one run (ms)						Speed-up of the whole run					
			Allo-	S1, S2,			Stages,	one row per message			pixel by pixel		
Image	Size	Read	cate	S3	Write	Total	share	Thr.	Pipes	Shm	Thr.	Pipes	Shm
1	450 × 180	3.6	7.5	3.3	2.0	16.3	20 %	1.63	1.55	1.78	0.30	0.33	0.24
4	1280 × 910	77	101	50	31	260	19 %	1.57	1.43	1.52	0.37	0.32	0.25
7	3750 × 2470	456	828	399	222	1905	21 %	1.56	1.41	1.42	0.26	0.28	0.22

Table 12: The experiment of Lab 1: one run of S1, S2 and S3 (variant 1), median of 5 runs (9 for image 1). "Allocate" is the time to create the three result images, one block per pixel.

With one run, the three stages are a fifth of the work. The assignment expected the reading and writing of the files to dominate, and they do: the stages are only 19 to 21 % of a run of the baseline. The rest is reading the file byte by byte (22 to 30 %), writing it (12 %) and, the largest part, allocating the three result images with one `new` per pixel (39 to 46 %). Three cores can only shorten the fifth that the stages take, by a factor of at most 1.6 to 1.9, which would make the whole run about 1.1 times faster.

The measured gain is larger, but it is not parallelism. With one row per message the programs finish 1.4 to 1.8 times sooner than the baseline. Most of this comes from something else: a pipeline stage keeps what it receives in a compact array, so these programs allocate only one image of per-pixel blocks (the output) instead of three. The stages themselves took 0.25 s in the thread program against 0.40 s in the baseline for image 7. Pixel by pixel, a single run is 3 to 5 times slower than the baseline, for the reasons given above.

6 Ease of Implementing and Debugging

	Threads + atomic operations	Processes + pipes	Processes + shared memory + semaphores
Setting up	Nothing: globals are shared, a queue is a struct.	`pipe()` twice, `fork()` three times, close the unused ends.	`shm_open`, `ftruncate`, `mmap`, `shm_unlink`, four `sem_init`, then `fork()`. The most code.
Synchronization we write	All of it: the lock, and what to do when the queue is full or empty.	None. The kernel blocks the reader and the writer.	The order of the four `sem_wait`/`sem_post` calls and the two initial values.
What can go wrong	Data races (everything is shared), a forgotten lock, false sharing, wasted cores.	A write end left open, short reads and writes, both sides must agree on the message size.	A wrong initial value or a missing `sem_post`: every process sleeps for ever. `pshared = 0`. A leftover object in `/dev/shm`.
If one stage dies	The whole process dies.	The neighbours get an end-of-file or `SIGPIPE` and stop.	The other stages wait on their semaphores for ever.
Debugging	One process in `gdb`, but the bugs depend on timing and can vanish under the debugger.	Three processes: attach `gdb` to one, or follow the child of a `fork`. Their prints interleave.	As for pipes, and a deadlock shows only as three sleeping processes.

Table 13: The three approaches compared.

Pipes were the easiest to get right. There is no shared state at all: a stage owns its data, and the only way to influence another stage is to write bytes into a pipe. All the difficult parts (blocking, waking up, buffering) are inside the kernel and already correct. The pipe program worked at the first attempt. A stage that fails takes the others down cleanly, which makes mistakes visible instead of leaving a hung program.

Threads were the quickest to write and the hardest to trust. The stages are ordinary functions and the queue is twenty lines. But every variable is shared, so nothing protects us from our own mistakes. We removed the lock on purpose to see what happens: the program did not crash, it simply never finished, in six attempts out of six. `count++` and `count--` are not atomic, so an update is lost whenever both sides run them at the same moment. When the stuck program was inspected, `count` said 64 (full) in both queues although only 5 to 49 messages were really in them, so the senders were waiting for room that nobody would ever announce. The false-sharing problem of Section 3 was worse, because the output was correct and the check passed; only the timing showed that something was wrong, and the cause was the placement of variables in memory, which the source code does not show.

Shared memory with semaphores needed the most care. It combines the setup work of processes with the shared state of threads. The synchronization is short but has to be exactly right, and a mistake does not give a wrong result but a deadlock without any message. Pointers need attention too: the address stored in the shared queue is valid in the children only because the memory is mapped before `fork()`. In return it is the mechanism that copies the least: a message

is copied once into the shared slot and once out, without entering the kernel as long as nobody has to sleep.

The proof helped with debugging. Because every link checks its own count and hash, a failure points at one link and one direction instead of at "the output is wrong". It also checks all 1000 runs, whereas the output file shows only the last one.

7 Part III: Two Computers

Design. Each computer runs one single-threaded program, and the stages that share a computer run one after the other on it. Computer A is the server: it reads the image, calls `getaddrinfo` with `AI_PASSIVE`, then `bind`, `listen` and `accept`. Computer B is the client: `getaddrinfo` with the address of A, then `connect` (it retries for 30 s, so the order in which the two are started does not matter much). This follows the example of the `getaddrinfo` man page with one deliberate change: the example uses UDP datagrams (`SOCK_DGRAM`), we use TCP (`SOCK_STREAM`). A datagram may be lost, duplicated or overtaken by a later one, and our stream carries no coordinates, so one lost datagram would shift every pixel after it. TCP gives exactly what a link of the pipeline needs: the bytes arrive once, in the order in which they were sent.

Protocol. A sends a header (width, height, variant, runs and `BATCH` as 32-bit integers in network byte order), then the input image once (243 KB), then the results as in Part II, and finally its pixel count and hash. B checks them against its own, and answers with one byte when its last run is finished. A stops its clock when this byte arrives, so the processing time covers the work of both computers. Computer B needs no file and only one argument that matters, the address of A; it even takes the image size from the header.

Only one computer was available. We could not run the two programs on two computers, so both ran on the same laptop as two processes, connected through the loopback address 127.0.0.1. The programs are unchanged for two computers: only `HOST` is different. The measurements below therefore show how the *work* is divided between two workers, but not what a real network adds, which is discussed at the end.

			`part3_1`: A: S1+S2, B: S3		`part3_2`: A: S1, B: S2+S3	
Variant	`BATCH`	Sent from A to B (MB)	Time (s)	Speed-up	Time (s)	Speed-up
1	1	243	45.1	0.063	47.0	0.061
1	16	243	4.52	0.63	5.89	0.49
1	450	243	2.57	1.11	2.25	1.27
1	81 000	243	2.46	1.16	2.07	1.38
2	1	2.4	2.43	1.18	2.22	1.29
2	450	2.4	2.30	1.25	1.98	1.45
Limit set by the more loaded computer				1.24		1.61

Table 14: Part III, image 1, 1000 runs, both programs on one computer. Median of 3 runs (5 for variant 2 with `BATCH=1`). The baseline takes 2.86 s.

The second partition is better. In every row where communication does not dominate, *S1 | S2+S3* is faster than *S1+S2 | S3*: 2.25 against 2.57 s with one row per message, 2.07 against 2.46 s with whole images, and 1.98 against 2.30 s in variant 2. The reason is the balance of the work, not the network, because both partitions send exactly the same number of bytes (3 per pixel). S1 is 62 % of the work. In the first partition computer A gets S2 on top of it and carries 81 % of the work, while B has only the 20 % of S3 and waits most of the time. In the second partition A has 62 % and B 38 %. The pipeline is as fast as the more loaded computer, so the limits are 1.24 and 1.61

(Section 2), and the measured speed-ups, 1.11–1.25 and 1.27–1.45, stay just below them. The best partition is the one that makes the two loads as equal as possible; for this application that means giving the heavy stage a computer of its own.

Pixel by pixel, the partition does not matter. With `BATCH=1` in variant 1 both partitions take about 46 s, 16 times longer than the baseline: one `write()` and one `read()` per pixel cost far more than any stage. They are still faster than the three-process programs of Part II in the same situation (57 and 82 s), simply because there is one link instead of two.

What a real network would change. The sockets were not given `TCP_NODELAY`, so the kernel collects small writes into full packets (Nagle's algorithm) and a 3-byte message does not become a 3-byte packet. The amount of data is what matters. Variant 1 sends 243 MB. Over the loopback interface this is nothing, but a 100 Mbit/s link carries 12.5 MB per second, so the transfer alone would need at least 19 s, ten times longer than the computation, and the network would be the slowest "stage" whichever partition is used. Variant 2 sends 2.4 MB (plus the 0.24 MB input image), about 0.2 s on the same link, and would keep most of its speed-up. On two real computers the choice of the partition therefore matters for variant 2 and for fast networks, and the amount of data sent matters more than anything else on slow ones. These last numbers are estimates from the amount of data, not measurements.

8 Comparison with the Results of Assignment 1

Reference. The Lab 1 folder still contains the images that the Lab 1 program wrote for its seven test images (`1o.ppm` to `7o.ppm`). We first compiled the unchanged Lab 1 source again and confirmed that it reproduces all seven files exactly, so they are a valid reference. Since every run of S1 starts from the same input image, every program of this assignment must produce exactly these files, whatever the variant, the number of runs or `BATCH`.

The snippet. `compare_outputs.sh` (also `make compare`) compares each of the eleven output files with the Lab 1 output, byte by byte:

```
for output in $expected_outputs; do
    if [ ! -f "$output" ]; then
        printf "%-30s : not created yet\n" "$output"
    elif cmp -s "$lab1_output" "$output"; then
        printf "%-30s : IDENTICAL to the Lab 1 output\n" "$output"
    else
        # cmp -l prints one line for every byte that differs
        bytes=$(cmp -l "$lab1_output" "$output" 2>/dev/null | wc -l)
        printf "%-30s : DIFFERENT (%s differing bytes)\n" "$output" "$bytes"
    fi
done
```

`run_all_parts.sh` runs all eleven `make` targets on an image (the two programs of Part III as two processes) and then calls the snippet.

Result. Every output of every part is identical to the original Lab 1 output: the same size and not one differing byte. Table 15 lists what was compared. The first row is what the makefile does by default; the other rows repeat the test on all seven Lab 1 images and with message sizes that stress the code in different ways (997 is a prime, so the last message of every image is a partial one).

Runs	BATCH	Outputs identical to Lab 1 (of 11), for image							Outputs
		1	2	3	4	5	6	7	
1000	1 (the make defaults)	11	–	–	–	–	–	–	11
100	1	11	11	–	–	–	–	–	22
100	997	11	11	11	11	11	11	11	77
100	whole image	11	11	11	–	–	–	–	33
Total: identical / different									143 / 0

Table 15: Comparison with the original outputs of Lab 1 (`1o.ppm` to `7o.ppm`; the sizes of the images are in Table 11). The 11 outputs are `part1` and both variants of `part2_1`, `2_2`, `2_3`, `3_1` and `3_2`. In the same runs all 260 link checks of Section 4 passed.

In addition, each of the 576 timed runs of Section 5 compared its output with the Lab 1 output of its image before the time was recorded, and all were identical. For the default targets the snippet prints `IDENTICAL to the Lab 1 output` for each of the eleven files and ends with `11 identical, 0 different, 0 not created yet`; the reference file has 243 015 bytes and the MD5 sum `cf3bb55d59f95b3f9a79c3f4a71ebfc0`.

9 Conclusion

The same three-stage pipeline was built with threads and atomic operations, with pipes, with shared memory and semaphores, and with sockets. All of them are correct in the strongest sense we could test: every output is identical to the Lab 1 output, and on every link the receiver's count and order-dependent hash equal the sender's, for all 81 million pixels of a run.

The main result is about the *granularity* of communication, and it is the same for all four mechanisms. Passing each pixel on its own costs 0.5 to 1 μs per pixel against 35 ns to compute it, so the "parallel" programs are 16 to 28 times slower than the baseline. Passing only every 100th result (variant 2), or the same pixels in messages of a few hundred, makes the cost of communication disappear and all programs become faster than the baseline, by a factor of 1.1 to 1.6. A factor of about 1.25 of this does not come from the extra cores: the stages of the pipeline store their intermediate pixels more compactly than the Lab 1 program does, and are faster for that reason alone. With large messages the mechanisms can hardly be told apart. With small ones the spin lock is the cheapest per message, while pipes and semaphores need the kernel to put a waiting stage to sleep and to wake it up; on the other hand busy waiting keeps cores occupied that sleeping stages leave free.

The speed-up cannot go beyond 1.61 with this decomposition, because S1 is 62 % of the work and the other two stages wait for it most of the time. The same number decides Part III: *S1 | S2+S3* (limit 1.61) is better than *S1+S2 | S3* (limit 1.24). To use three cores fully, the heavy stage itself would have to be divided, for example by giving each core a third of the rows, instead of giving each core a stage.

Pipes were the easiest mechanism to get right, threads the easiest to write and the hardest to trust, and shared memory with semaphores needed the most care. The two hardest problems were not logic errors at all, but a cache line shared by accident and a function placed at an unlucky address. Both left the output correct and only showed up when the parts were timed separately.